

데이터베이스 구축 개요

학습목표

1. 데이터베이스 설계의 개요
2. 개체-관계 데이터 모델
3. 데이터베이스 설계 예

- 데이터 과학자가 IT 전문가와 협의, 협력 필요
- IT 전문가는 DBA(DB 설계자), 응용 프로그래머
- 데이터 요청 시 DB 내용을 파악 중요
- 협의 시 DB 관계자가 DB 설계도 사용 가능
- DB구조를 표현하는 개념 집합, 데이터 모델
- 사용 후보 모델은 개념 데이터 모델, 표현 데이터 모델
- 어떤 설계도 형식도 이해 가능 능력 중요
- 어떤 과정으로 DB가 구축되는지 사전 파악이 도움

0.1 DB 설계자의 역할

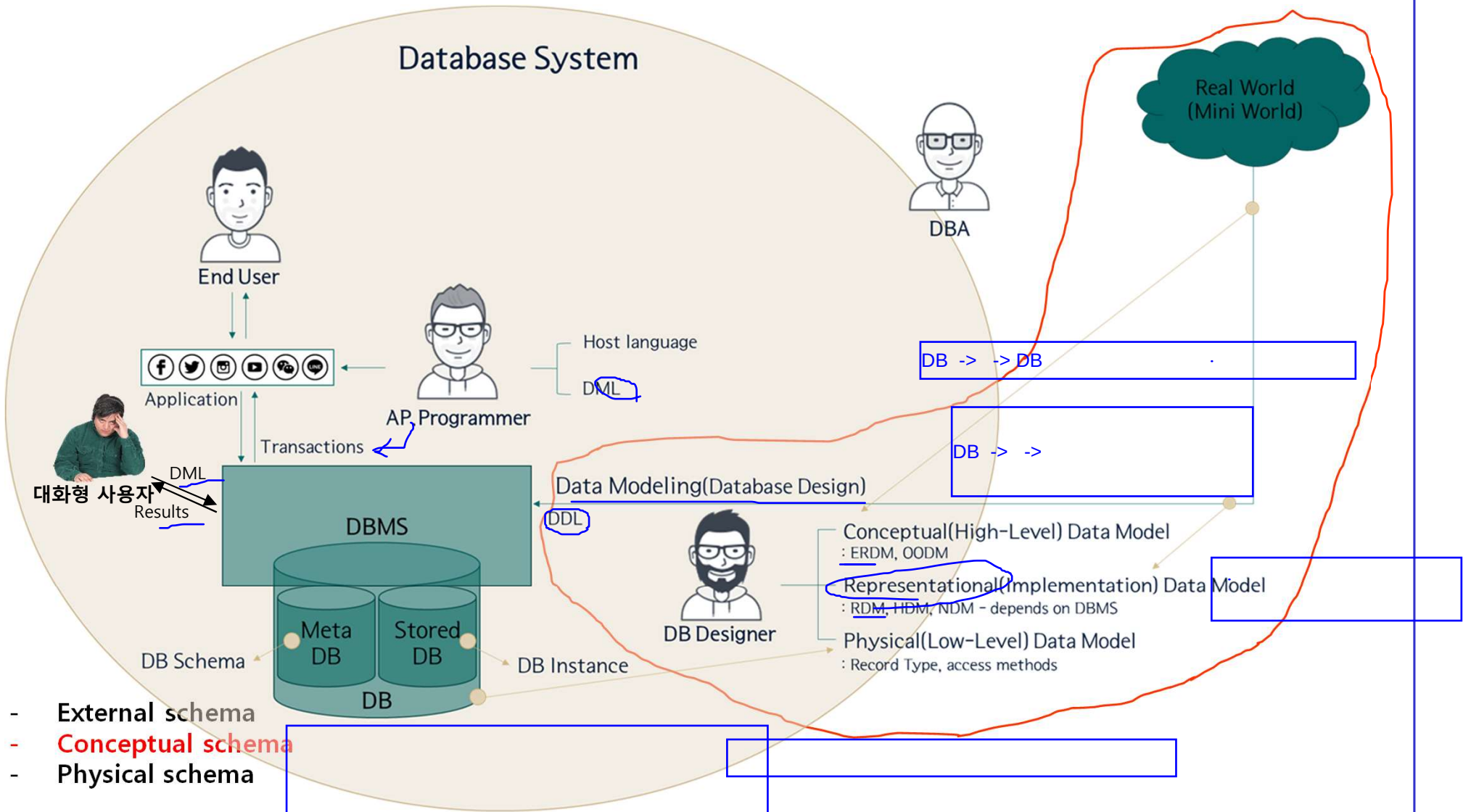


Figure1.1-1 Database users for database system.

0.2 데이터 모델의 종류

□ 데이터 모델의 분류

✓ 고수준 또는 개념적 데이터 모델(conceptual data model)

- 사람이 인식하는 것과 유사하게 데이터베이스의 전체적인 논리적 구조를 명시
- 예: 개체-관계(ER: Entity-Relationship) 데이터 모델과 객체 지향 데이터 모델

✓ 표현(구현) 데이터 모델(representation(implementation) data model)

- 최종 사용자가 이해하는 개념이면서 컴퓨터 내에서 데이터가 조직되는 방식과 멀리 떨어져 있지는 않음
- 예: 계층 데이터 모델(hierarchical data model), 네트워크 데이터 모델(network data model), 관계 데이터 모델(relational data model)

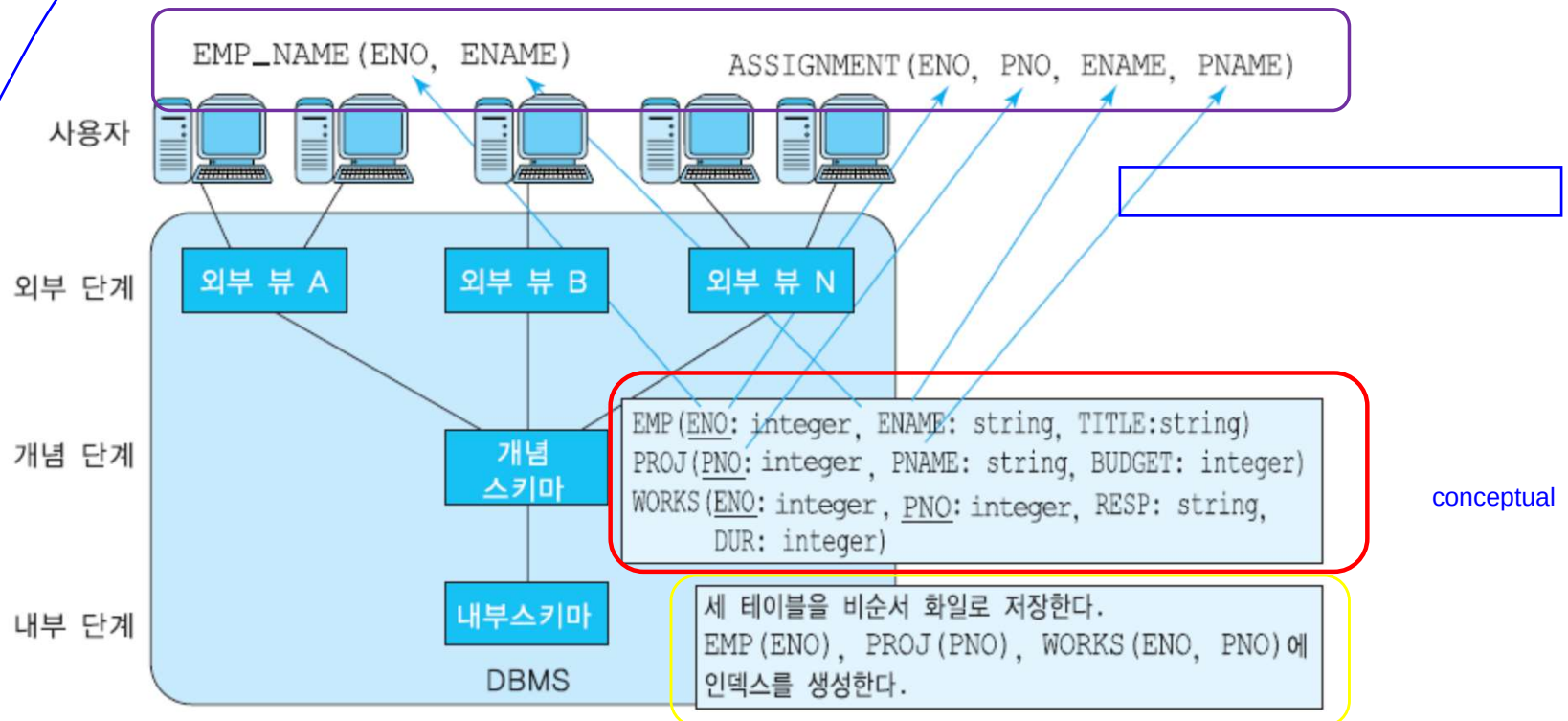
✓ 저수준 또는 물리적인 데이터 모델(physical data model)

- 데이터베이스에 데이터가 어떻게 저장되는가를 기술
- 예: Unifying, ISAM, VSAM 등

0.3 ANSI/SPARC 3-레벨 아키텍처

예 :

그림 1.19는 관계 데이터 모델에서 개념 단계의 세 테이블 EMP, PROJ, WORKS로부터 일부 애플리케이션을 선택하여 외부 단계의 뷰 ASSIGNMENT와 EMP_NAME이 정의된 예를 보여준다.



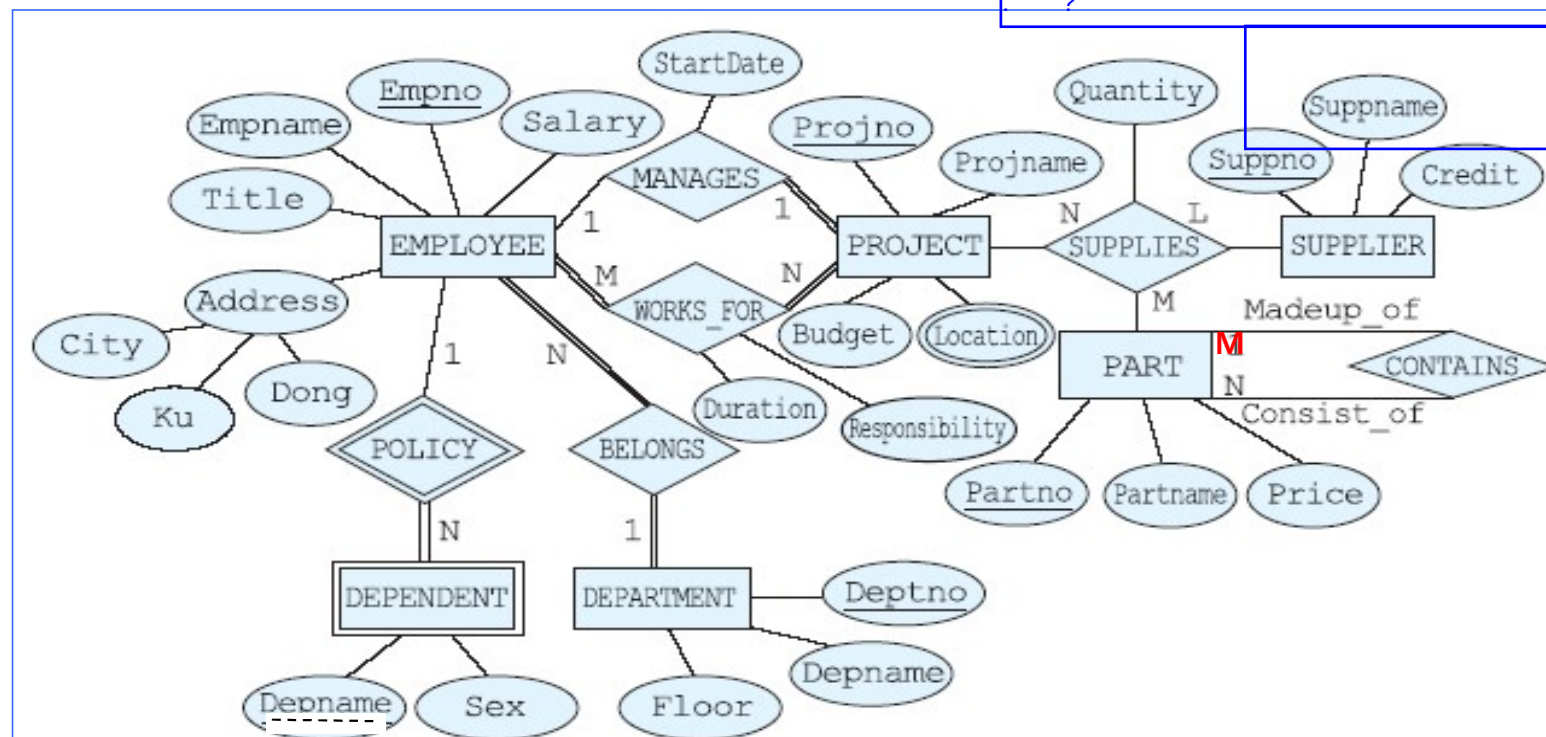
[그림 1.19] 관계 데이터 모델에서 3단계 아키텍처의 예

0.4 필요 DB에 대한 정보 예

□ 필요한 DB에 대해 DBA와 협의 시 만날 수 있는 설계도

✓ 필요한 사항을 정확하게 전달하기 위해 설계도 이해 필수

✓ 설계도를 기반으로 필요한 정보 존재 여부 파악 및 구체적 요청 가능



[그림 5.42] 회사의 ER 스키마 다이어그램

1. 데이터베이스 설계 개요

□ 데이터베이스 설계

- ✓ 개념적 데이터베이스 설계와 물리적 데이터베이스 설계로 구분
- ✓ 개념적 데이터베이스 설계는 실제로 데이터베이스를 어떻게 구현할 것인가와는 독립적으로 정보 사용의 모델을 개발하는 과정 ?
- ✓ 물리적 데이터베이스 설계에서는 물리적인 저장 장치와 접근 방식을 다룸
- ✓ 개념적 데이터베이스 설계 과정에서 조직체(실세계)의 엔티티, 관계, 프로세스, 무결성 제약조건 등을 나타내는 추상화 모델을 구축
- ✓ 엔티티(entity, 개체)는 서로 구분이 되면서 조직체에서 데이터베이스에 나타내려는 객체(사람, 장소, 사물 등)를 의미
- ✓ 관계는 두 개 이상의 엔티티들 간의 연관을 나타냄
- ✓ 무결성 제약조건은 데이터의 정확성과 비즈니스 규칙을 의미

1. 데이터베이스 설계 개요

□ 개념적 수준의 모델

- ✓ 특정 **표현(구현) 데이터 모델**과 독립적으로 응용 세계를 모델링할 수 있도록 함
- ✓ 데이터베이스 구조를 **하향식**으로 개발할 수 있기 위한 틀/framework)을 제공함
- ✓ 인기 있는 개념적 수준의 모델은 **개체-관계(ER: Entity-Relationship) 모델**
- ✓ ER 모델과 같은 개념적인 데이터 모델이 사상될 수 있는 다수의 **표현(구현) 데이터 모델**(representation/implementation data model)이 존재함
- ✓ 표현/구현 단계에서 사용되는 세 가지 데이터 모델: **관계 데이터 모델**, 계층 데이터 모델, 네트워크 데이터 모델

1. 데이터베이스 설계 개요

□ 데이터베이스 설계의 개요

- ✓ 한 조직체의 운영과 목적을 지원하기 위해 데이터베이스를 생성하는 과정
- ✓ 목적은 모든 주요 응용과 사용자들이 요구하는 데이터, 데이터 간의 관계를 표현하는 것
- ✓ 훌륭한 데이터베이스 설계는 시간의 흐름에 따른 데이터의 모든 측면을 나타내고, 데이터 항목의 중복을 최소화하고, 데이터베이스에 대한 효율적인 접근을 제공하고, 데이터베이스의 무결성을 제공하고, 현실 세계를 자연스럽게 표현하여 이해하기 쉬워야 함

1. 데이터베이스 설계 개요

〈표 5.1〉 데이터베이스 개발의 라이프 사이클

데이터베이스 설계 과정

단계	기능	질문
요구사항 분석 단계	요구사항 수집과 분석	
설계 단계		
개념적 설계	ER 모델링 또는 객체 지향 모델	어떤 엔티티와 관계들이 요구되는가?
DBMS의 선정		어떤 DBMS가 적절한가?
논리적 설계	개념적 설계를 데이터베이스 스키마로 사상	데이터베이스가 무엇을 모델링하는가?
스키마 정제	중복을 제거함 - 데이터베이스 스키마의 정규화	가장 단순한 스키마?
물리적 설계	성능상의 문제를 고려하여 인덱스 등을 정의	어떤 성능을 원하는가?
보안 설계	사용자들의 그룹과 접근 제한	어떤 수준의 보안을 원하는가?
구현 단계		
	데이터베이스의 구축과 튜닝	

데이터를 적재하거나 변환

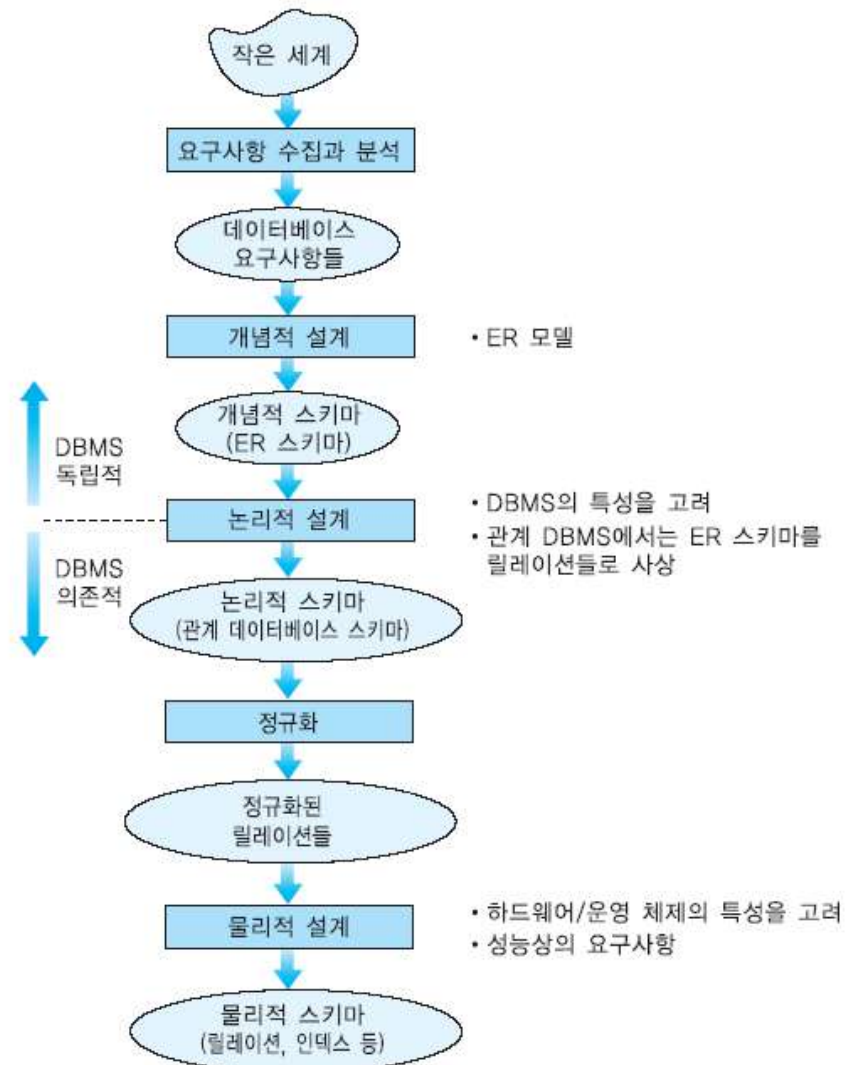
기존의 응용 변환

1. 데이터베이스 설계 개요

□ 데이터베이스 설계의 주요 단계

- ✓ 요구사항 분석, 개념적 설계, DBMS의 선정, 논리적 설계(스키마 정제), 물리적 설계와 튜닝 등 여러 작업들로 이루어짐
- ✓ 일반적으로, 데이터베이스 설계의 완성도를 높이기 위해서 이런 작업들을 앞뒤로 왔다갔다할 필요가 있음

1. 데이터베이스 설계 개요



[그림 5.1] 데이터베이스 설계의 주요 단계

1. 데이터베이스 설계 개요

□ 요구사항 수집과 분석

- ✓ 흔히 기존의 문서를 조사하고, 인터뷰나 설문 조사 등이 시행됨
- ✓ 요구사항에 관한 지식을 기반으로 관련 있는 엔티티들과 이들의 애트리뷰트들이 무엇인가, 엔티티들 간의 관계가 무엇인가 등을 파악함
- ✓ 또한 데이터 처리에 관한 요구사항에 대하여 전형적인 연산들은 무엇인가, 연산들의 의미, 접근하는 데이터의 양 등을 분석함

1. 데이터베이스 설계 개요

□ 개념적 설계

- ✓ 모든 물리적인 사항과 독립적으로, 한 조직체에서 사용되는 정보의 모델을 구축하는 과정
- ✓ 사용자들의 요구사항 명세로부터 개념적 스키마가 만들어짐
- ✓ 높은 추상화 수준의 데이터 모델을 기반으로 정형적인 언어(그림)로 데이터 구조를 명시함
- ✓ 대표적인 데이터 모델이 ER 모델
- ✓ 개념적 설계의 단계에서는 엔티티 타입, 관계 타입, 애트리뷰트들을 식별하고, 애트리뷰트들의 도메인을 결정하고, 후보 키와 기본 키 애트리뷰트들을 결정함
- ✓ 완성된 개념적 스키마(ER 스키마)는 ER 다이어그램으로 표현됨

1. 데이터베이스 설계 개요

□ DBMS 선정

- ✓ 여러 가지 요인들을 검토한 후 DBMS를 선정함
- ✓ 기술적인 요인은 DBMS가 제공하는 데이터 모델, 데이터 저장/접근 구조, 인터페이스, 질의어, 도구, 제공되는 서비스 등
- ✓ 경제적인 요인은 DBMS 구입 비용, 하드웨어 구입 비용, 유지 보수(서비스) 비용, 기존의 시스템을 새로운 DBMS에 맞게 변환하는데 소요되는 비용, 인건비, 교육비 등

1. 데이터베이스 설계 개요

□ 논리적 설계

- ✓ 데이터베이스 관리를 위해 선택한 DBMS의 데이터 모델을 사용하여 논리적 스키마(외부 스키마도 포함)를 생성함
- ✓ 논리적 스키마를 나타내기 위해 관계 데이터 모델을 사용하는 경우에는, ER 모델로 표현된 개념적 스키마를 관계 데이터베이스 스키마로 사상함
- ✓ 관계 데이터베이스 스키마를 더 좋은 관계 데이터베이스 스키마로 변환하기 위해서 정규화(normalization) 과정을 적용함
- ✓ 데이터베이스 설계자가 요구사항 수집과 분석 후에 바로 논리적 설계 단계로 가는 경우가 있는데, 이런 경우에는 흔히 좋은 관계 데이터베이스 스키마가 생성되지 않음

1. 데이터베이스 설계 개요

□ 물리적 설계

- ✓ 처리 요구사항들을 만족시키기 위해 저장 구조와 접근 경로 등을 결정함
- ✓ 성능상의 주요 기준은 몇 가지로 구분할 수 있음
 - **응답 시간**: 질의와 갱신이 평균적으로 또는 피크 시간 때 얼마나 오래 걸릴 것인가?
 - **트랜잭션 처리율**: 온라인 응용에서 1초당 얼마나 많은 트랜잭션들이 평균적으로 또는 피크 시간 때 처리될 수 있는가?

2. ER 모델

□ ER 모델

- ✓ 데이터베이스 설계를 용이하게 하기 위해서 P.P. Chen이 1976년에 제안하였음
- ✓ 그 후에 많은 학자들이 이 모델을 강화시켰음
- ✓ 현재는 EER(Enhanced(Extended) Entity-Relationship)이 데이터베이스 설계 과정에 널리 사용되고 있음
- ✓ 개념적 설계를 위한 인기 있는 모델로서, 높은 수준으로 추상화하며, 이해하기 쉬우며, 구문들의 표현력이 뛰어나고 사람들이 응용에 대해 생각하는 방식과 가깝고, 많은 CASE 도구들에서 지원됨
- ✓ 실세계를 엔티티, 애트리뷰트, 엔티티들 간의 관계로 표현함
- ✓ 쉽게 관계 데이터 모델로 사상됨

2. ER 모델

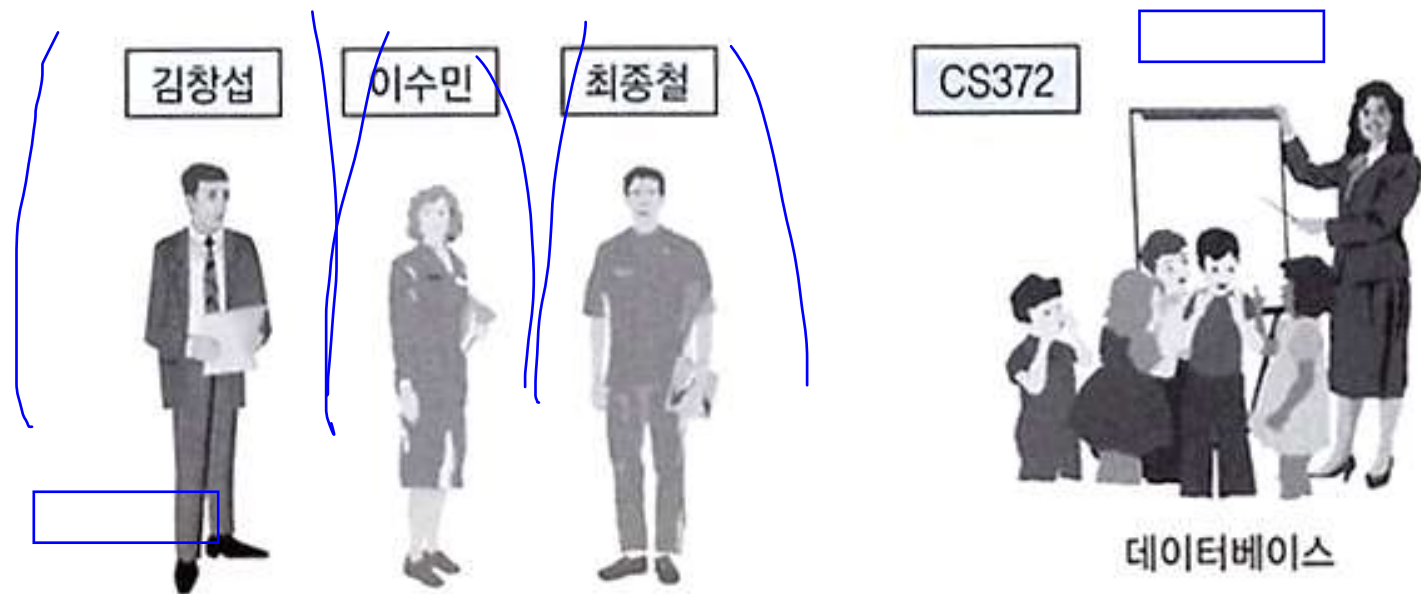
□ ER 모델(계속)

- ✓ 기본적인 표현 내용(구문)으로는 엔티티, 관계, 애트리뷰트(키 선정)가 있고, 기타 내용으로는 카디날리티 비율, 참여 제약조건 등이 있음
- ✓ 적은 노력으로 쉽게 배울 수 있고, 전문가가 아니어도 이해하기 쉬우며, 자연어보다는 좀더 정형적이고, 구현에 독립적이어서 데이터베이스 설계자들이 최종 사용자와 의사 소통을 하는데 적합함
- ✓ ER 모델을 기반으로 만들어진 다수의 CASE 도구(예, **ERWin** 등)들이 존재함
- ✓ 이런 도구들은 ER 설계를 자동적으로 SQL Server, 오라클, 사이베이스 등의 데이터 정의어로 변환하고, 어떤 도구는 XML로 변환함

2. ER 모델

□ 엔티티

- ✓ 하나의 엔티티는 사람, 장소, 사물, 사건 등과 같이 독립적으로 존재하면서 고유하게 식별이 가능한 실세계의 객체
- ✓ 사원처럼 실체가 있는 것도 있지만 생각이나 개념과 같이 추상적인 것도 있음



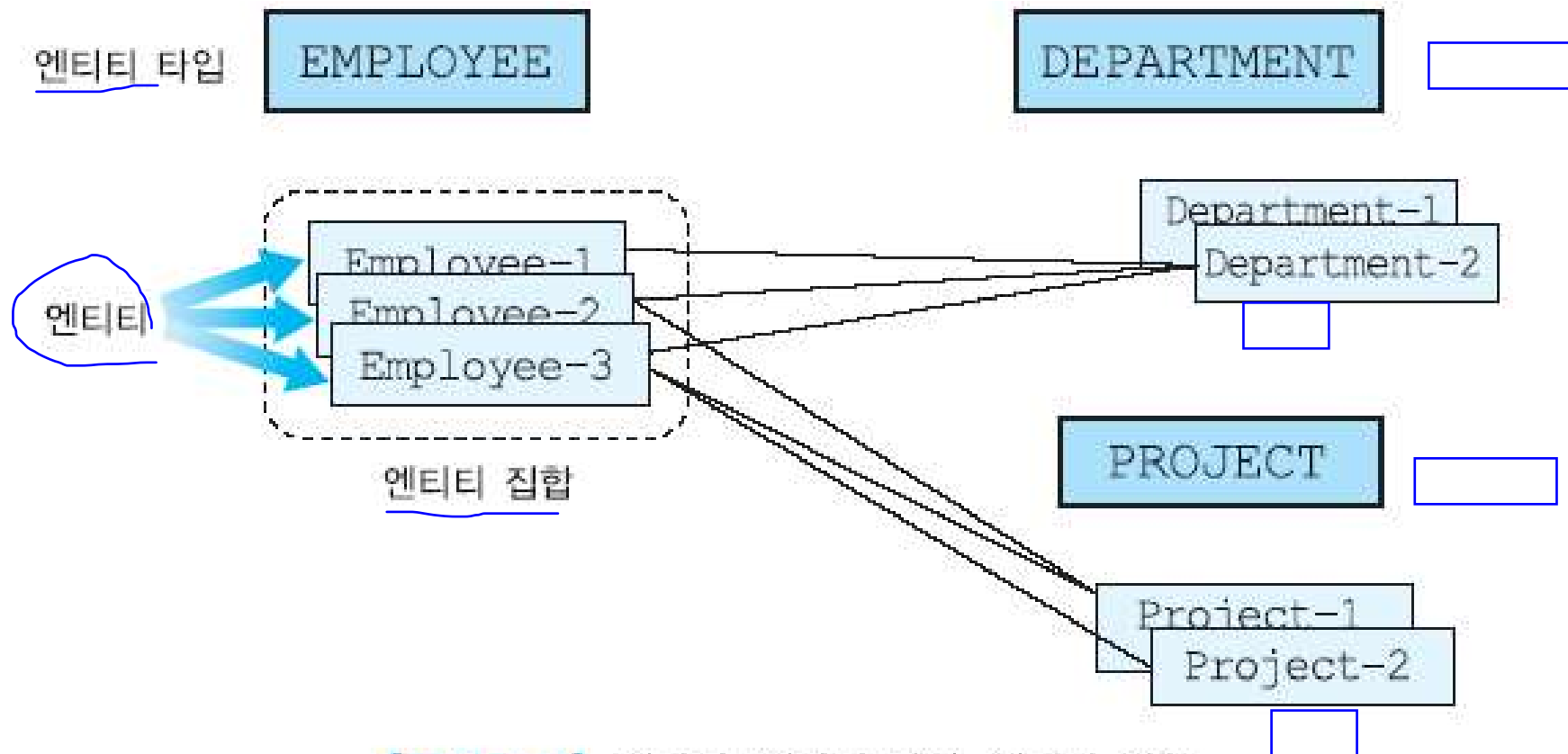
[그림 5.2] 엔티티의 예

2. ER 모델

□ 엔티티 타입

- ✓ 같은 종류의 엔티티들이 많이(최소 2개 이상) 존재하며, 엔티티들은 엔티티 타입(또는 엔티티 집합)들로 분류됨
- ✓ 엔티티 집합은 동일한 애트리뷰트들을 가진 엔티티들의 모임
- ✓ 엔티티 타입은 동일한 애트리뷰트들을 가진 엔티티들의 틀
- ✓ 하나의 엔티티는 한 개 이상의 엔티티 집합에 속할 수 있음
- ✓ 엔티티 집합과 엔티티 타입을 엄격하게 구분할 필요는 없음
- ✓ ER 다이어그램에서 엔티티 타입은 직사각형으로 나타냄

2. ER 모델



[그림 5.3] 엔티티, 엔티티 타입, 엔티티 집합

2. ER 모델

□ 강한 엔티티 타입 (그냥 엔티티 타입이라고 함)

- ✓ 강한 엔티티 타입(정규 엔티티 타입)은 독자적으로 존재하며 엔티티 타입 내에서 자신의 키 애트리뷰트를 사용하여 고유하게 엔티티들을 식별할 수 있는 엔티티 타입

□ 약한 엔티티 타입 (뒤에서 자세히 설명 예정)

- ✓ 약한 엔티티 타입은 키를 형성하기에 충분한 애트리뷰트들을 갖지 못한 엔티티 타입
- ✓ 이 엔티티 타입이 존재하려면 소유 엔티티 타입이 있어야 함
- ✓ 소유 엔티티 타입의 키 애트리뷰트를 결합해야만 고유하게 약한 엔티티 타입의 엔티티들을 식별할 수 있음

2. ER 모델

□ 애트리뷰트

- ✓ 하나의 엔티티는 연관된 애트리뷰트들의 집합으로 설명됨
 - 예: 사원 엔티티는 사원번호, 이름, 직책, 급여 등의 애트리뷰트를 가짐
- ✓ 한 애트리뷰트의 도메인은 그 애트리뷰트가 가질 수 있는 모든 가능한 값들의 집합을 의미
 - 예: 사원번호는 1000부터 9999까지의 값을 가짐
- ✓ 여러 애트리뷰트가 동일한 도메인을 공유할 수 있음
 - 예: 사원번호와 부서번호가 네 자리 정수를 가질 수 있음
- ✓ 키 애트리뷰트는 한 애트리뷰트 또는 애트리뷰트들의 모임으로서 한 엔티티 타입 내에서 각 엔티티를 고유하게 식별함
- ✓ ER 다이어그램에서 기본 키에 속하는 애트리뷰트는 밑줄을 그어 표시함

2. ER 모델

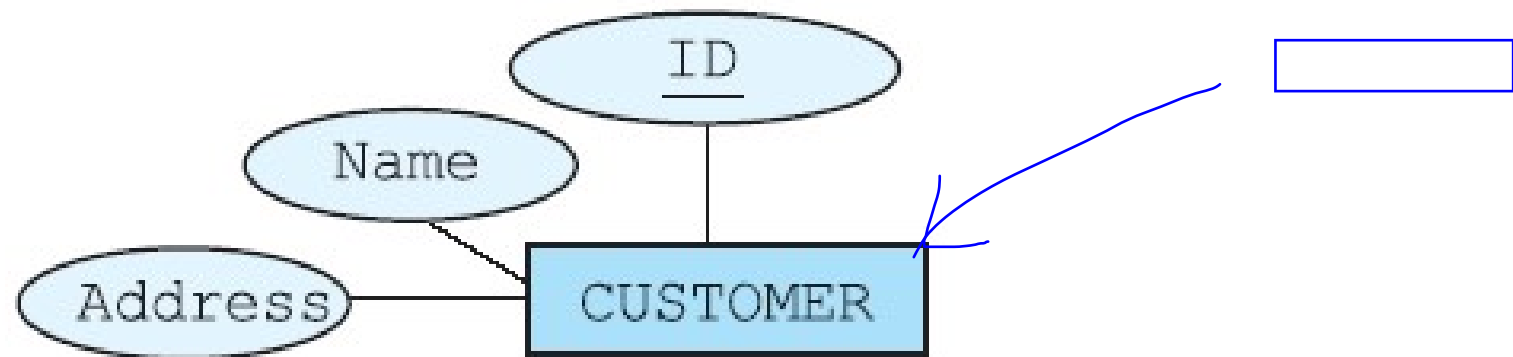
□ 애트리뷰트(계속)

- ✓ 요구사항 명세에서 명사나 형용사로 표현됨
- ✓ 엔티티는 독립적인 의미를 갖는데 반해서 애트리뷰트는 독립적인 의미를 갖지 않음
- ✓ ER 다이어그램에서 타원형으로 나타냄
- ✓ 애트리뷰트와 엔티티 타입은 실선으로 연결

2. ER 모델

□ 단순 애트리뷰트(simple attribute)

- ✓ 더 이상 다른 애트리뷰트로 나눌 수 없는(atomic) 애트리뷰트
- ✓ ER 다이어그램에서 **실선 타원**으로 표현함
- ✓ ER 다이어그램에서 대부분의 애트리뷰트는 단순 애트리뷰트



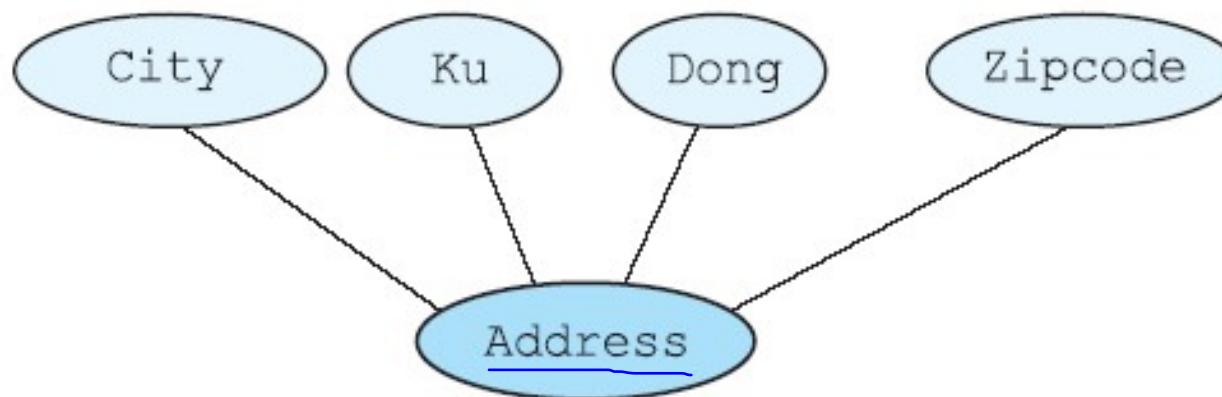
[그림 5.4] 단순 애트리뷰트

2. ER 모델

❑ 복합 애트리뷰트(composite attribute)

✓ 두 개 이상의 애트리뷰트로 이루어진 애트리뷰트

✓ 동일한 엔티티 타입이나 관계 타입에 속하는 애트리뷰트들 중에서 밀접하게 연관된 것을 모아놓은 것



[그림 5.5] 복합 애트리뷰트

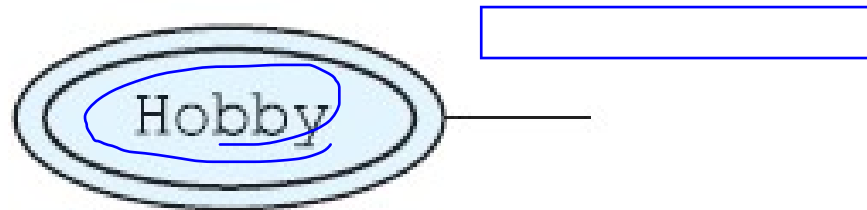
2. ER 모델

□ 단일 값 애트리뷰트(single-valued attribute)

- ✓ 각 엔티티마다 정확하게 하나의 값을 갖는 애트리뷰트
- ✓ ER 다이어그램에서 단순 애트리뷰트와 동일하게 표현됨
- ✓ 예: 사원의 사원번호 애트리뷰트는 어떤 사원도 두 개 이상의 사원번호를 갖지 않으므로 단일 값 애트리뷰트
- ✓ ER 다이어그램에서 대부분의 애트리뷰트는 단일 값 애트리뷰트

2. ER 모델

- ❑ 다치 애트리뷰트(multi-valued attribute)
 - ✓ 각 엔티티마다 여러 개의 값을 가질 수 있는 애트리뷰트
 - ✓ ER 다이어그램에서 이중선 타원으로 표현함



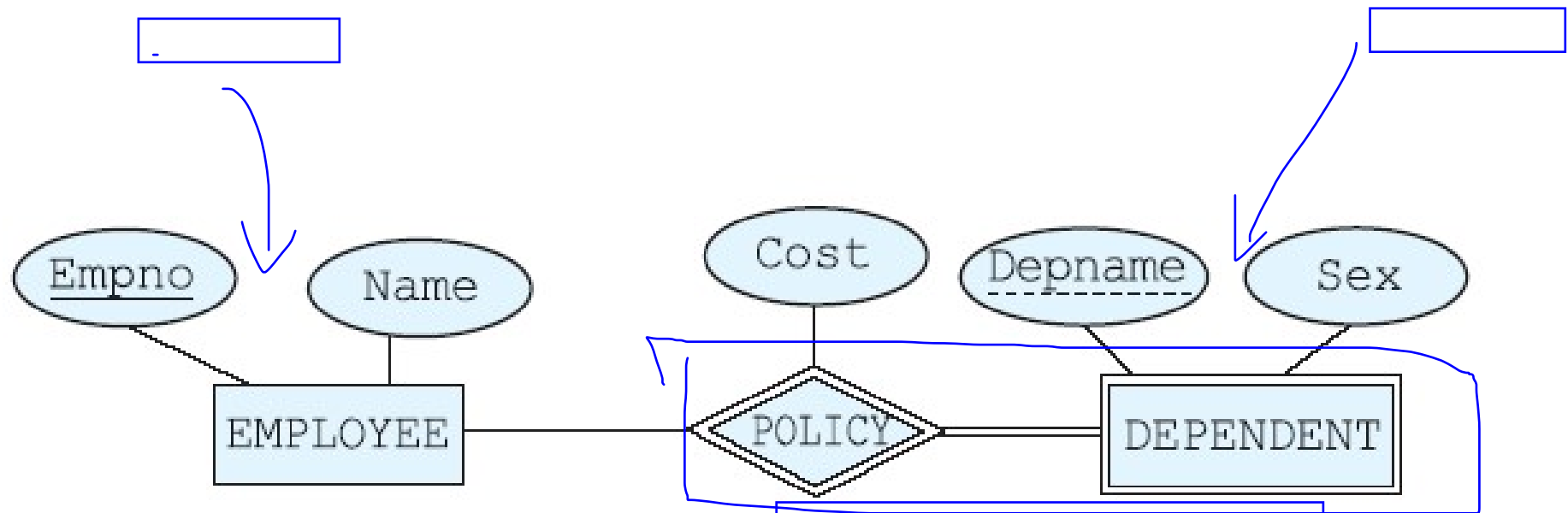
[그림 5.6] 다치 애트리뷰트

2. ER 모델

□ 약한(weak) 엔티티 타입

- ✓ 키를 형성하기에 충분한 애트리뷰트들을 갖지 못한 엔티티 타입
- ✓ 엔티티의 존재가 다른 엔티티의 존재에 의존적인 엔티티 집합을 위함
- ✓ 약한 엔티티 타입에게 키 애트리뷰트를 제공하는 엔티티 타입을 **소유 엔티티 타입**(owner entity type) 또는 **식별 엔티티 타입**(identifying entity type)라고 부름
- ✓ ER 다이어그램에서 이중선 직사각형으로 표기
- ✓ 약한 엔티티 타입의 부분 키는 점선 밑줄을 그어 표시
- ✓ 부분 키 (partial key): 부양가족의 이름처럼 한 사원에 속한 부양가족 내에서는 서로 다르지만 회사 전체 직원들의 부양가족들 전체에서는 같은 경우가 생길 수 있는 애트리뷰트

2. ER 모델



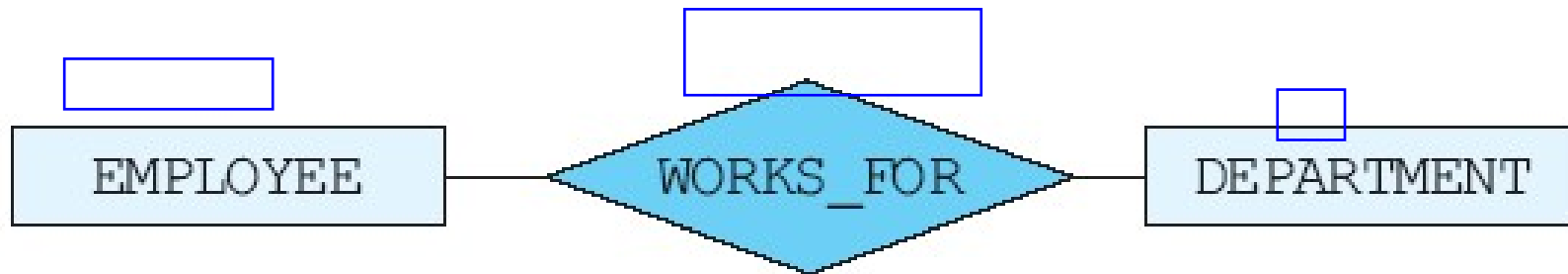
[그림 5.9] 약한 엔티티 타입

2. ER 모델

□ 관계와 관계 타입

- ✓ 관계는 엔티티들 사이에 존재하는 연관이나 연결로서 두 개 이상의 엔티티 타입들 사이의 사상으로 생각할 수 있음
- ✓ 관계 집합은 동질의 관계들의 집합
- ✓ 관계 타입은 동질의 관계들의 틀
- ✓ 관계 집합과 관계 타입을 엄격하게 구분할 필요는 없음
- ✓ 요구사항 명세에서 흔히 동사는 ER 다이어그램에서 관계로 표현됨
- ✓ ER 다이어그램에서 다이어몬드로 표기
- ✓ 관계 타입이 서로 연관시키는 엔티티 타입들을 관계 타입에 실선으로 연결함

2. ER 모델



[그림 5.10] 관계 타입 WORKS_FOR

〈표 5.2〉 엔티티와 엔티티 간의 관계의 예

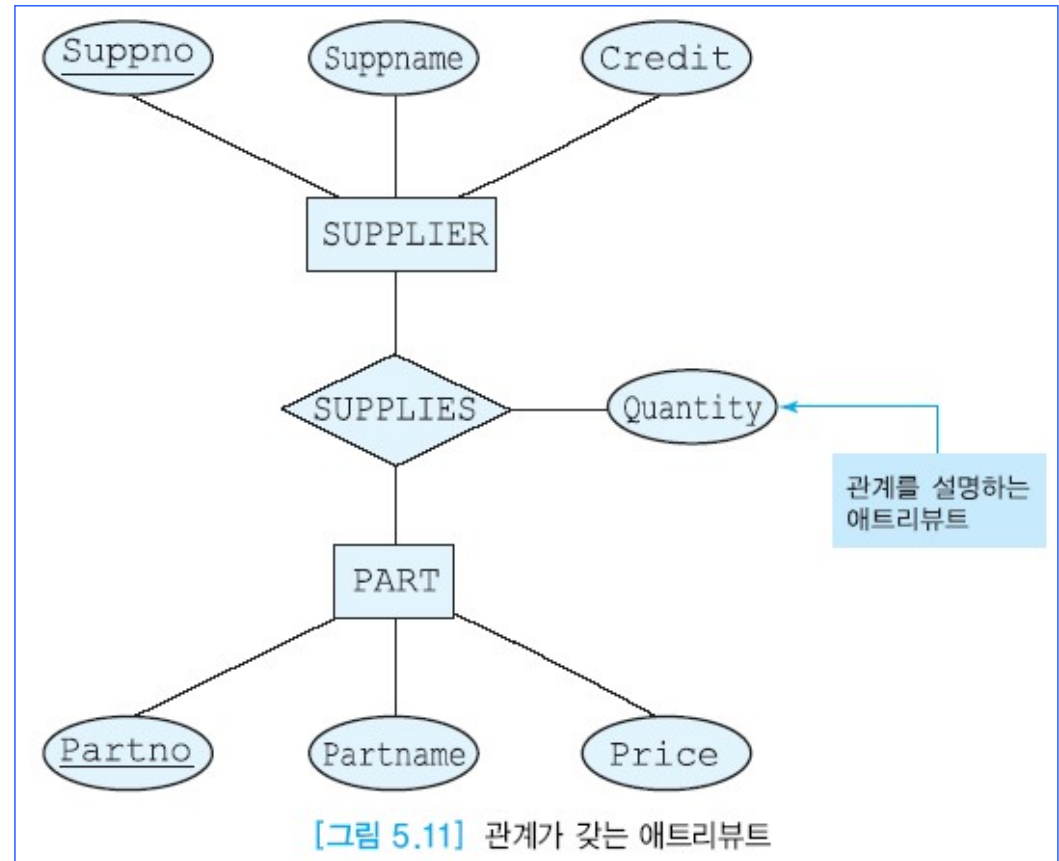
엔티티	관계	엔티티
사원(employee)	근무한다(works for)	부서(department)
공급자(supplier)	공급한다(supplies)	부품(part)
학생(student)	수강한다(enrolls)	과목(course)

2. ER 모델

□ 관계의 애트리뷰트

✓ 관계 타입은 관계의 특징을 기술하는 애트리뷰트들을 가질 수 있음

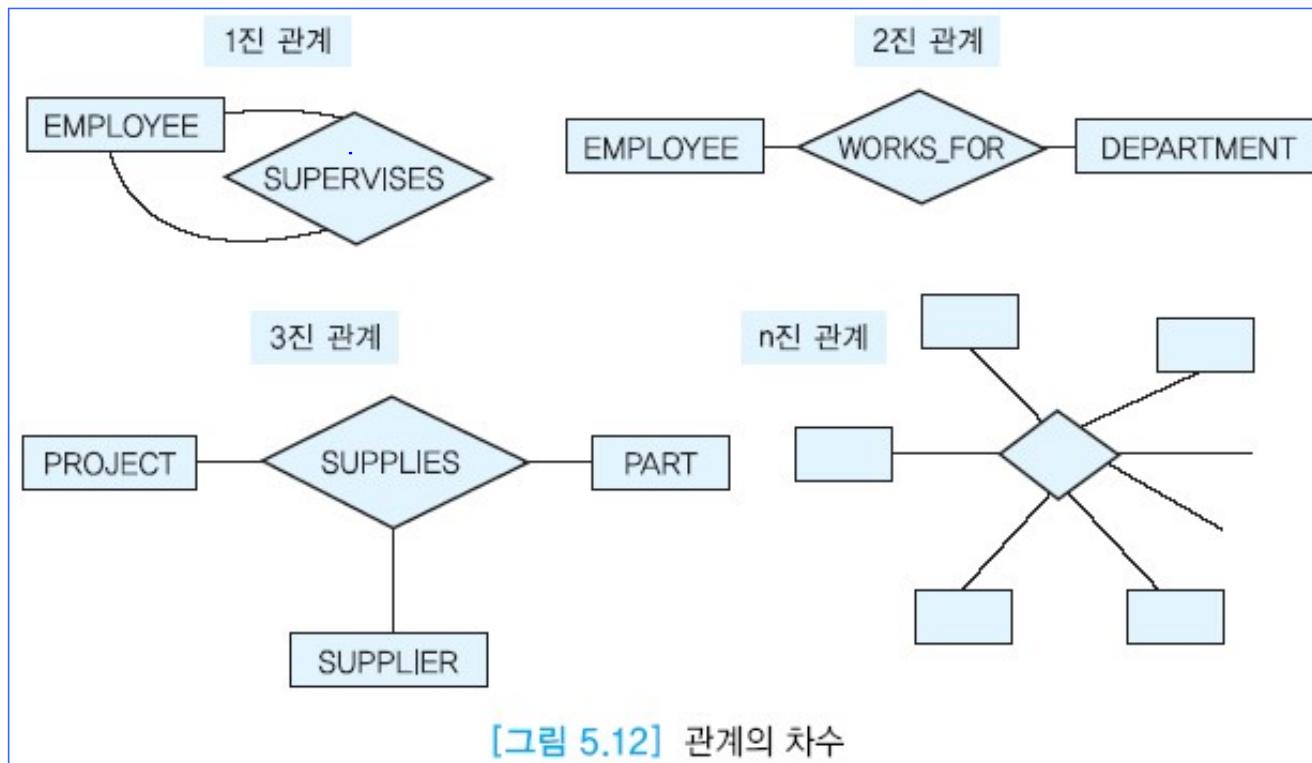
✓ 관계 타입은 키 애트리뷰트를 갖지 않음



2. ER 모델

□ 차수(degree)

- ✓ 관계로 연결된 엔티티 타입들의 개수를 의미
- ✓ 실세계에서 가장 흔한 관계는 두 개의 엔티티 타입을 연결하는 2진 관계

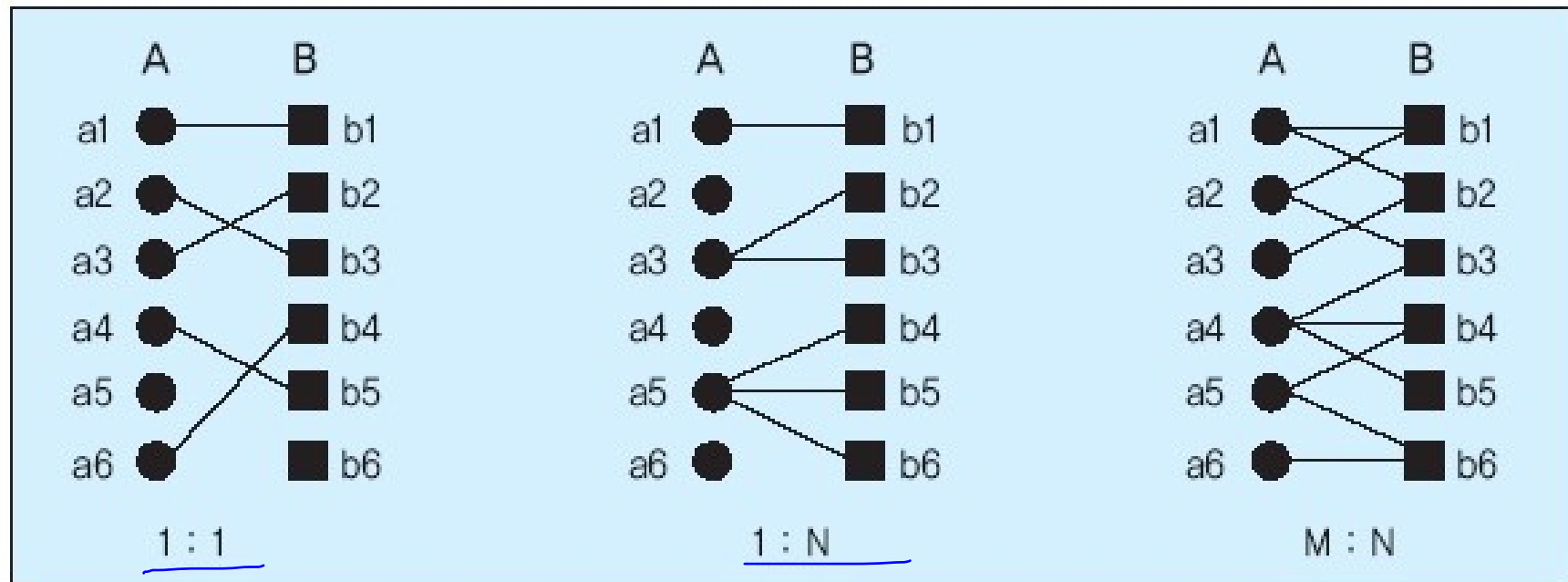


2. ER 모델

□ 카디날리티 (mapping cardinality, 대응수)

- ✓ 카디날리티 비율은 한 엔티티가 특정 관계 타입을 통해서 연관지을 수 있는 상대 엔티티 타입의 원소의 최대 수를 나타냄
- ✓ 관계 타입에 참여하는 엔티티들의 가능한 조합을 제한함
- ✓ 관계를 흔히 $1:1$, $1:N$, $M:N$ 으로 구분
- ✓ 카디날리티에 관한 정보는 간선 위에 나타냄

2. ER 모델



[그림 5.14] 카디날리티 비율

2. ER 모델

□ 전체 참여와 부분 참여

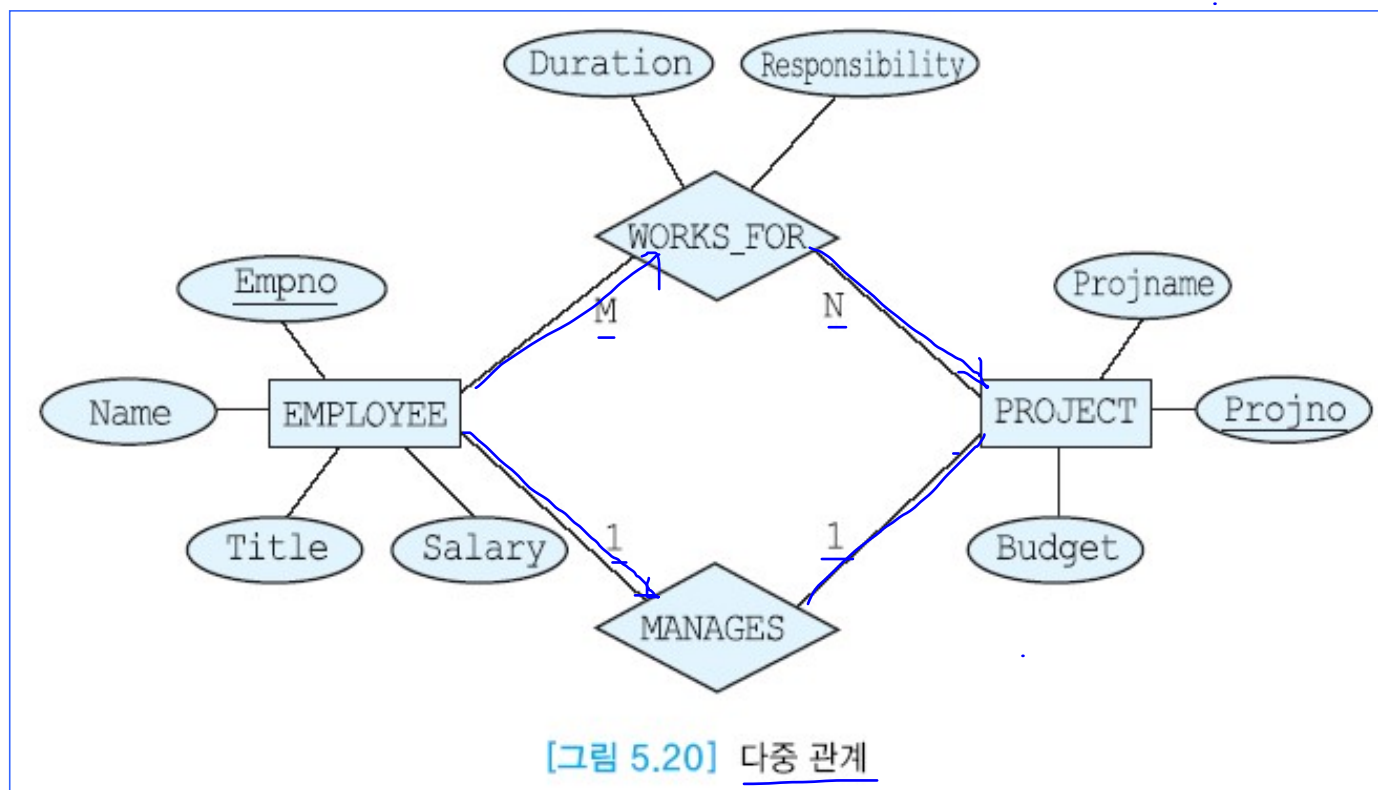
- ✓ 전체 참여는 어떤 관계에 엔티티 타입 E1의 모든 엔티티들이 관계 타입 R에 의해서 어떤 엔티티 타입 E2의 어떤 엔티티와 연관되는 것을 의미
- ✓ 부분 참여는 어떤 관계에 엔티티 타입 E1의 일부 엔티티만 참여하는 것을 의미
- ✓ 약한 엔티티 타입은 항상 관계에 전체 참여
- ✓ 전체 참여는 ER 다이어그램에서 **이중 실선**으로 표시
- ✓ 카디널리티 비율과 함께 참여 제약조건은 관계에 대한 중요한 제약조건



2. ER 모델

□ 다중 관계

✓ 두 엔티티 타입 사이에 두 개 이상의 관계 타입이 존재할 수 있음

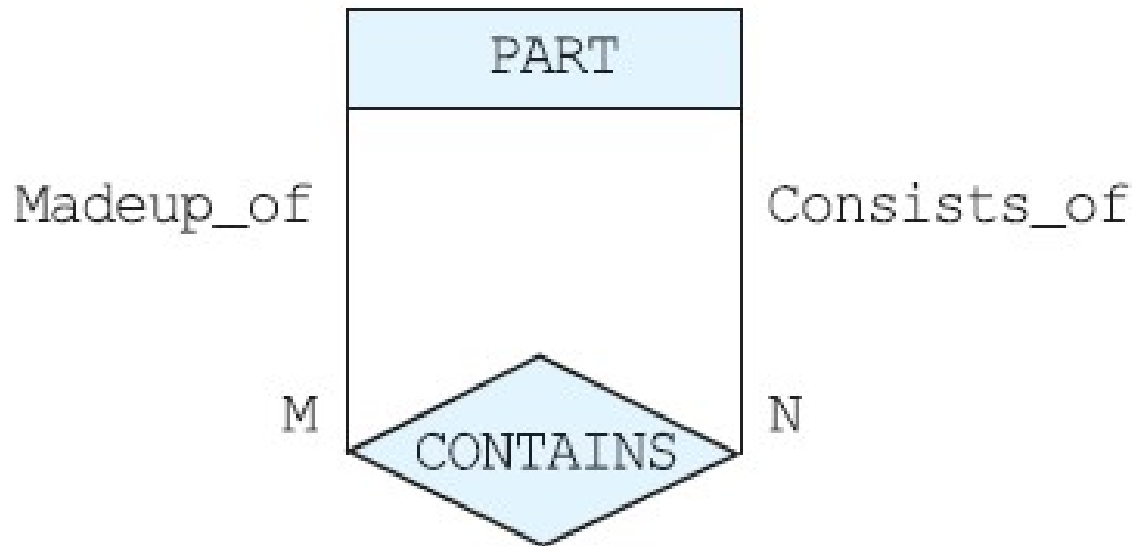


2. ER 모델

GPU :

□ 순환적 관계

- ✓ 하나의 엔티티 타입이 동일한 관계 타입에 두 번 이상 참여하는 것

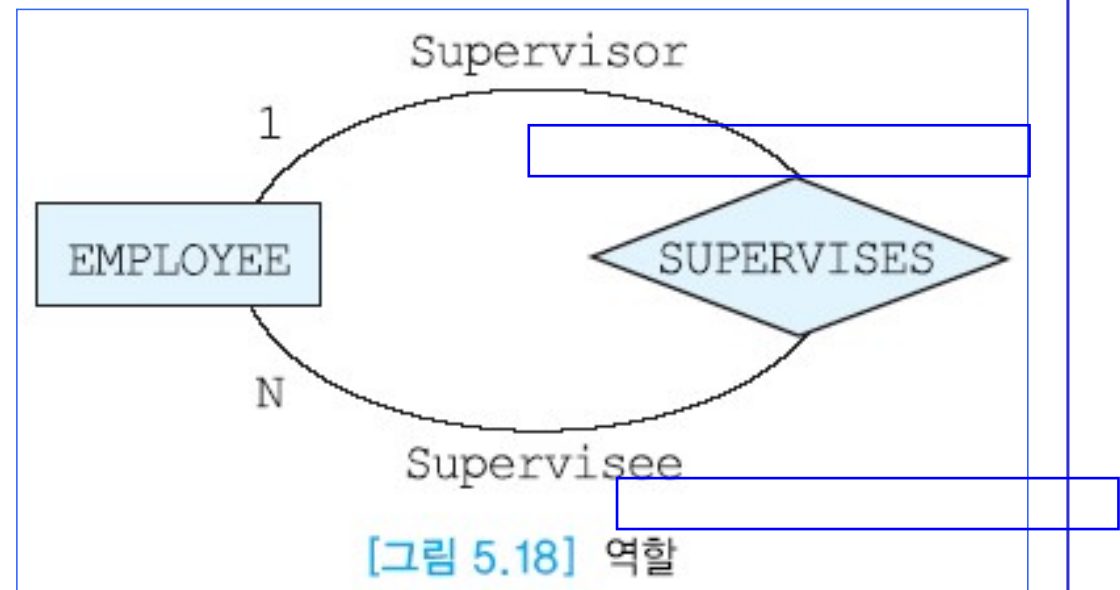


[그림 5.21] 순환적 관계

2. ER 모델

❑ 역할(role)

- ✓ 관계 타입의 의미를 명확하게 하기 위해 사용됨
- ✓ 특히 하나의 관계 타입에 하나의 엔티티 타입이 여러 번 나타나는 경우에는 반드시 역할을 표기해야 함 (순환적 관계)
- ✓ 관계 타입의 간선 위에 표시



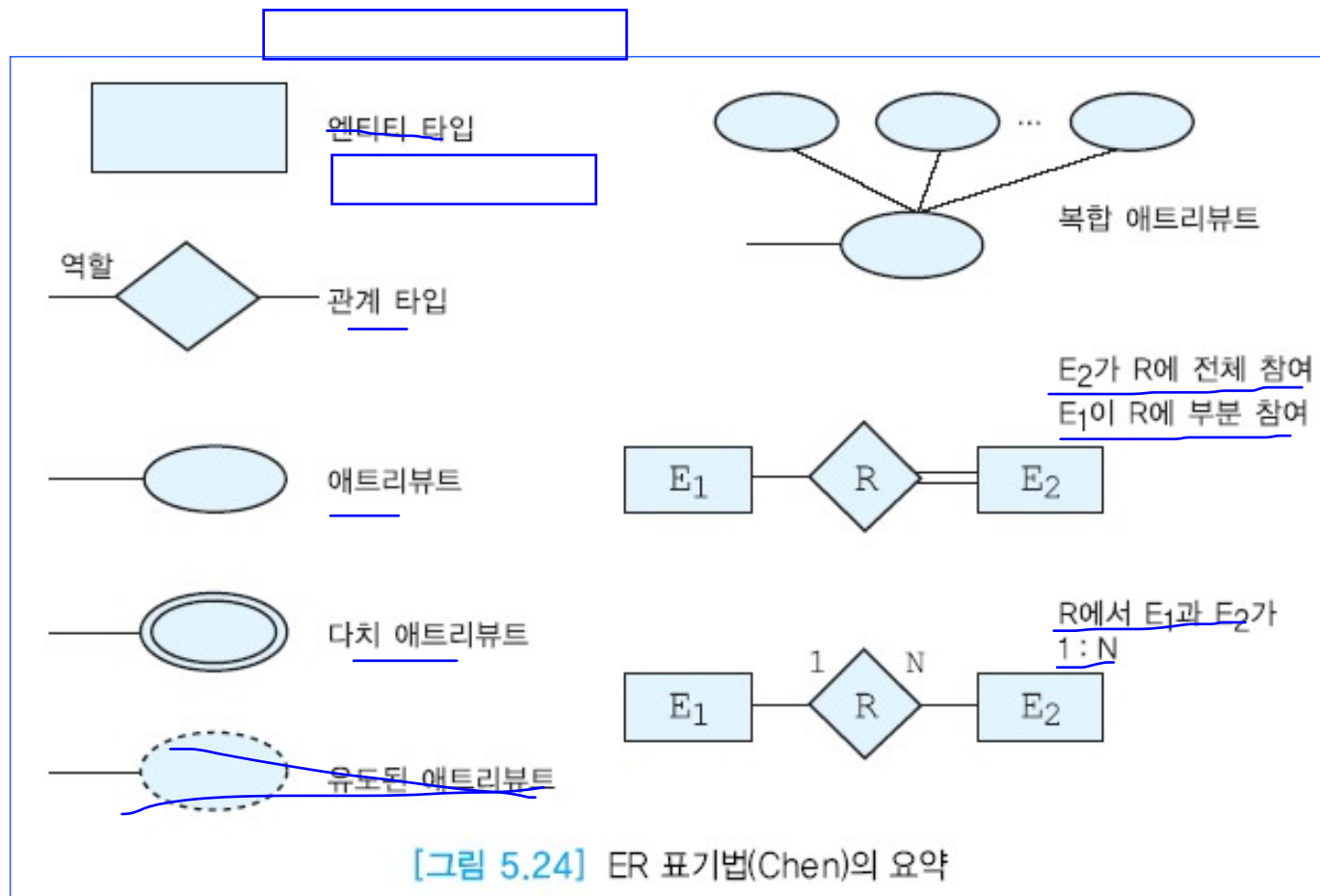
2. ER 모델

□ 데이터베이스 설계 과정 요약

- ✓ 응용의 요구사항을 수집하여 기술
- ✓ 응용과 연관이 있는 엔티티 타입들을 식별
- ✓ 응용과 연관이 있는 관계 타입들을 식별
- ✓ 관계가 1:1, 1:N, M:N 중에서 어느 것에 해당하는지 결정
- ✓ 엔티티 타입과 관계 타입들에 필요한 애트리뷰트들을 식별하고, 각 애트리뷰트가 가질 수 있는 값들의 집합을 식별
- ✓ 엔티티 타입들을 위한 기본 키를 식별
- ✓ 응용을 위한 ER 스키마 다이어그램을 그림
- ✓ ER 스키마 다이어그램이 응용에 대한 요구사항과 부합되는지 검사
- ✓ ER 스키마 다이어그램을 DBMS에서 사용되는 데이터베이스 모델로 변환

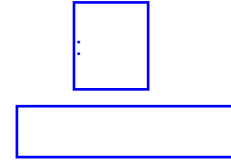
2. ER 모델

ER 표기법의 요약



3. 데이터베이스 설계 예

□ 기업에서 흔히 볼 수 있는 작은 세계에 관한 요구사항



- ✓ 회사에는 다수의 직원들이 재직
- ✓ 각 직원에 대해서 직원번호(고유함), 이름, 직책, 급여, 주소를 저장. 주소는 시, 구, 동으로 세분하여 나타냄
- ✓ 각 직원은 0명 이상의 부양가족을 가질 수 있음. 한 부양가족은 두 명 이상의 직원에게 속하지 않음. 각 부양가족에 대해서 부양가족의 이름과 성별을 저장
- ✓ 회사는 여러 개의 프로젝트들을 진행. 각 프로젝트에 대해서 프로젝트번호(고유함), 이름, 예산, 프로젝트가 진행되는 위치를 나타냄. 한 프로젝트는 여러 위치에서 진행될 수 있음. 각 프로젝트마다 여러 명의 직원들이 일함. 각 직원은 여러 프로젝트에서 근무할 수 있음. 각 직원이 해당 프로젝트에서 어떤 역할을 수행하고, 얼마 동안 근무해 왔는가를 나타냄. 각 프로젝트마다 한 명의 프로젝트 관리자가 있음. 한 직원은 두 개 이상의 프로젝트의 관리자가 될 수는 없음. 프로젝트 관리자 임무를 시작한 날짜를 기록

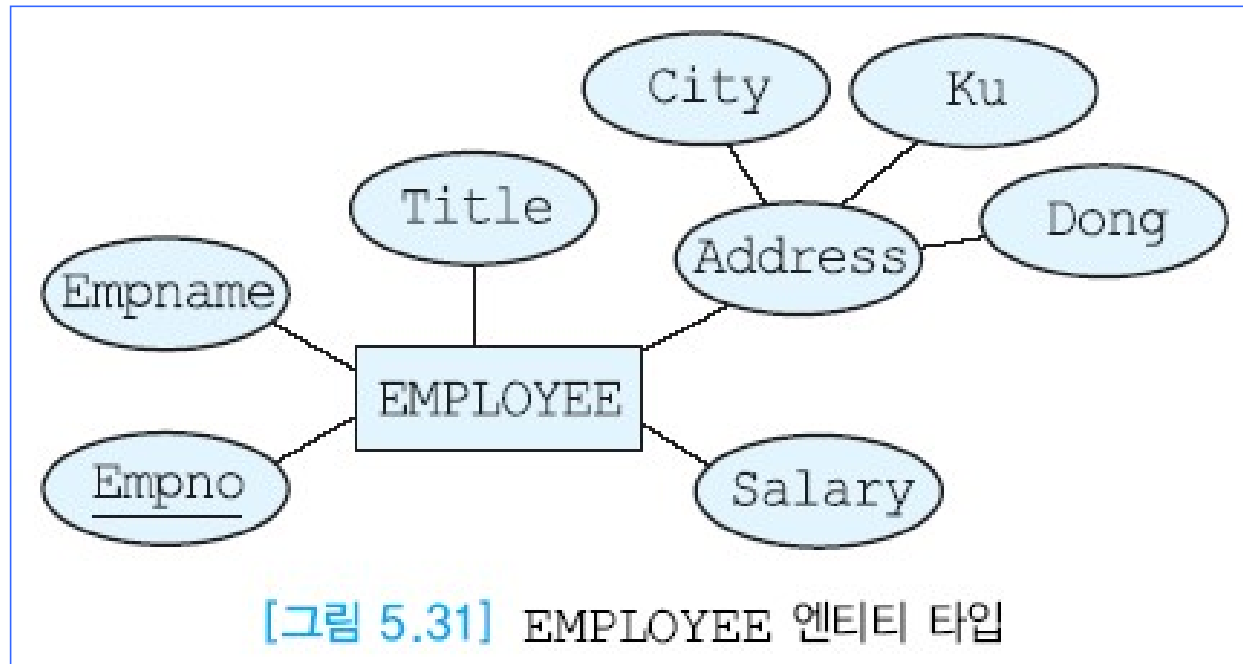
3. 데이터베이스 설계 예

□ 기업에서 흔히 볼 수 있는 작은 세계에 관한 요구사항(계속)

- ✓ 각 사원은 한 부서에만 속함. 각 부서에 대해서 부서번호(고유함), 이름, 부서가 위치한 층을 나타냄
- ✓ 각 프로젝트에는 부품들이 필요. 한 부품이 두 개 이상의 프로젝트에서 사용될 수 있음. 하나의 부품은 다른 여러 개의 부품들로 이루어질 수 있음. 각 부품에 대해서 부품번호(고유함), 이름, 가격, 그 부품이 다른 부품들을 포함하는 경우에는 그 부품들에 관한 정보도 나타냄
- ✓ 각 부품을 공급하는 공급자들이 있음. 한 명의 공급자는 여러 가지 부품들을 공급할 수 있고, 각 부품은 여러 공급자들로부터 공급될 수 있음. 각 공급자에 대해서 공급자번호(고유함), 이름, 신용도를 나타냄. 각 공급자에 대해서 그 공급자가 어떤 부품을 어떤 프로젝트에 얼마나 공급하는가를 나타냄

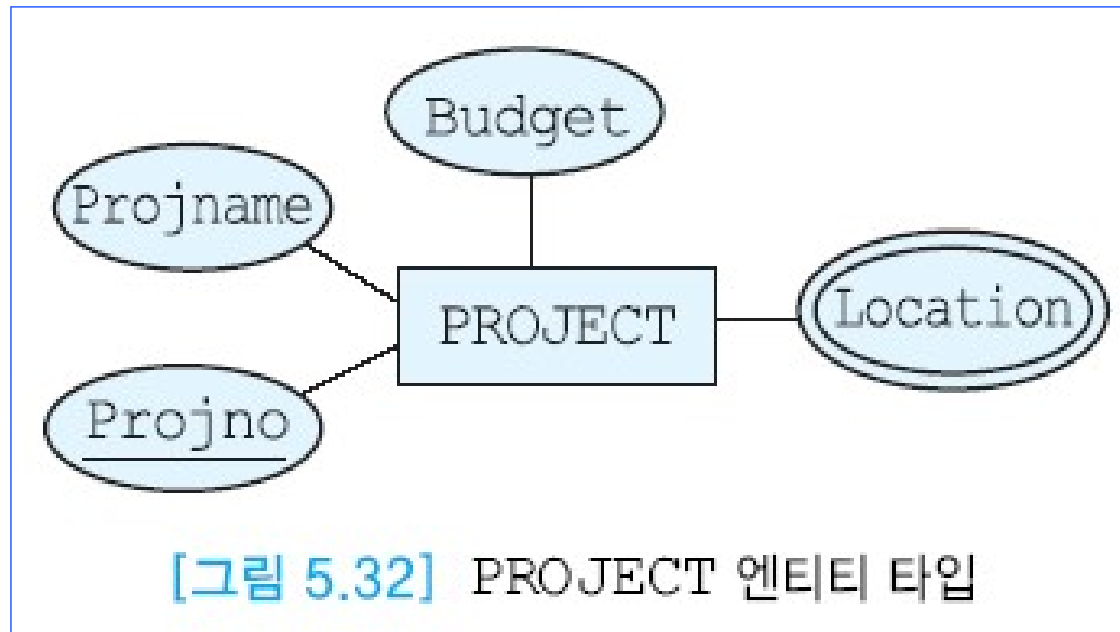
3. 데이터베이스 설계 예

□ 엔티티 타입 및 애트리뷰트들을 식별



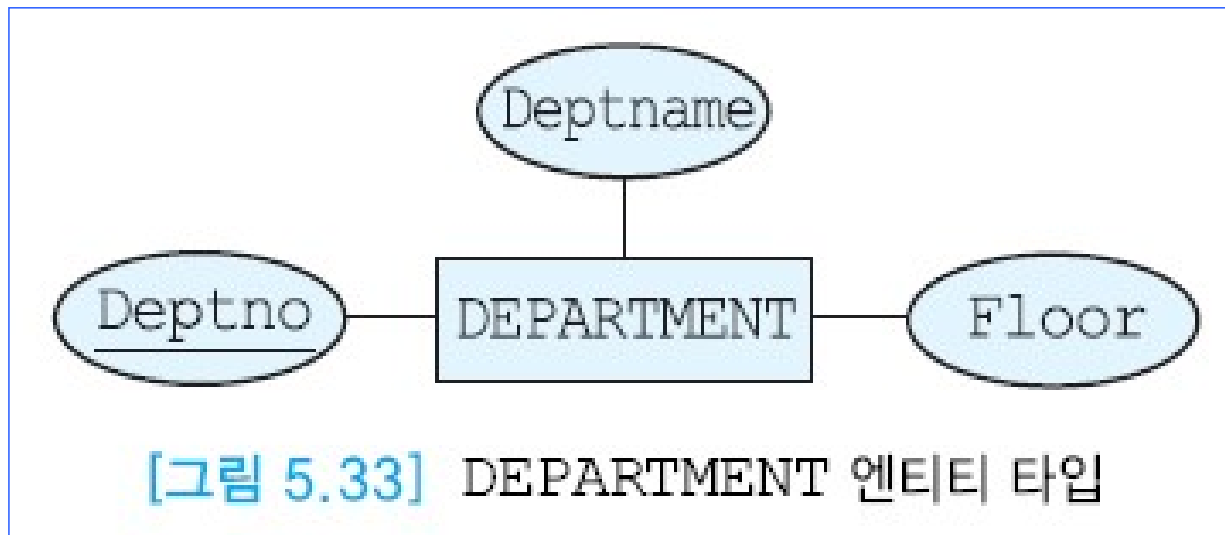
3. 데이터베이스 설계 예

□ 엔티티 타입 및 애트리뷰트들을 식별(계속)



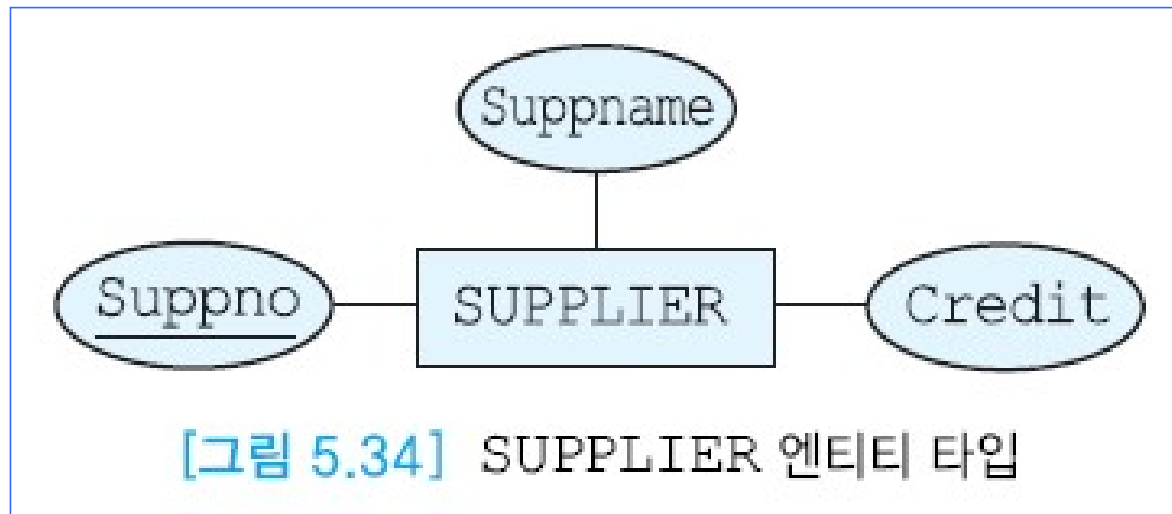
3. 데이터베이스 설계 예

□ 엔티티 타입 및 애트리뷰트들을 식별(계속)



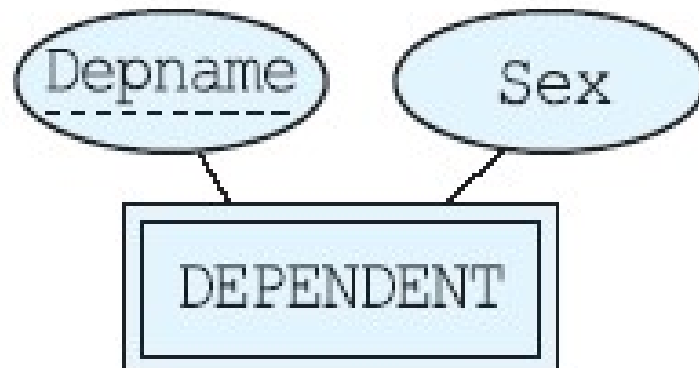
3. 데이터베이스 설계 예

□ 엔티티 타입 및 애트리뷰트들을 식별(계속)



3. 데이터베이스 설계 예

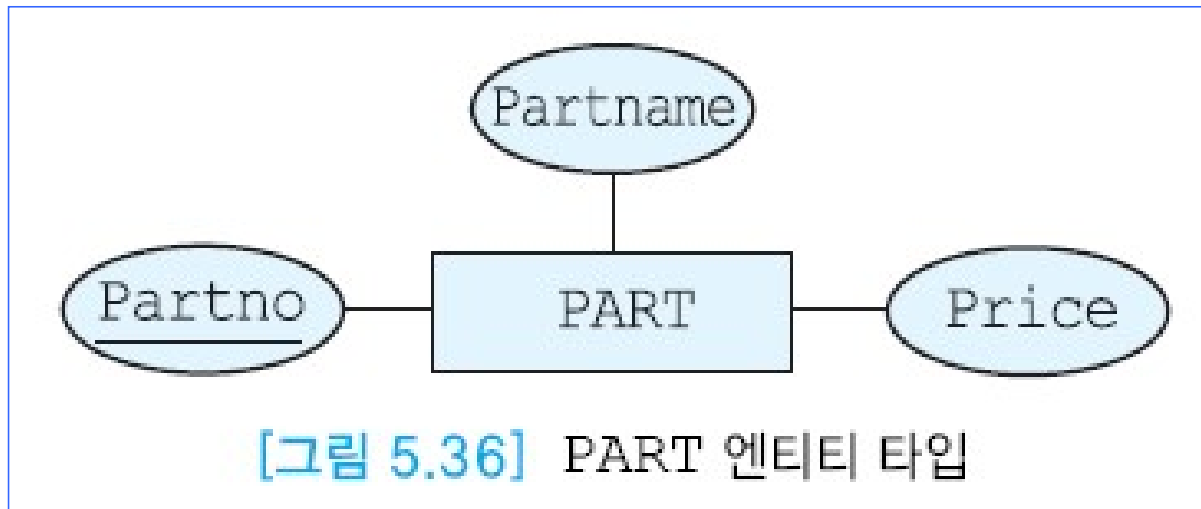
□ 엔티티 타입 및 애트리뷰트들을 식별(계속)



[그림 5.35] DEPENDENT 엔티티 타입

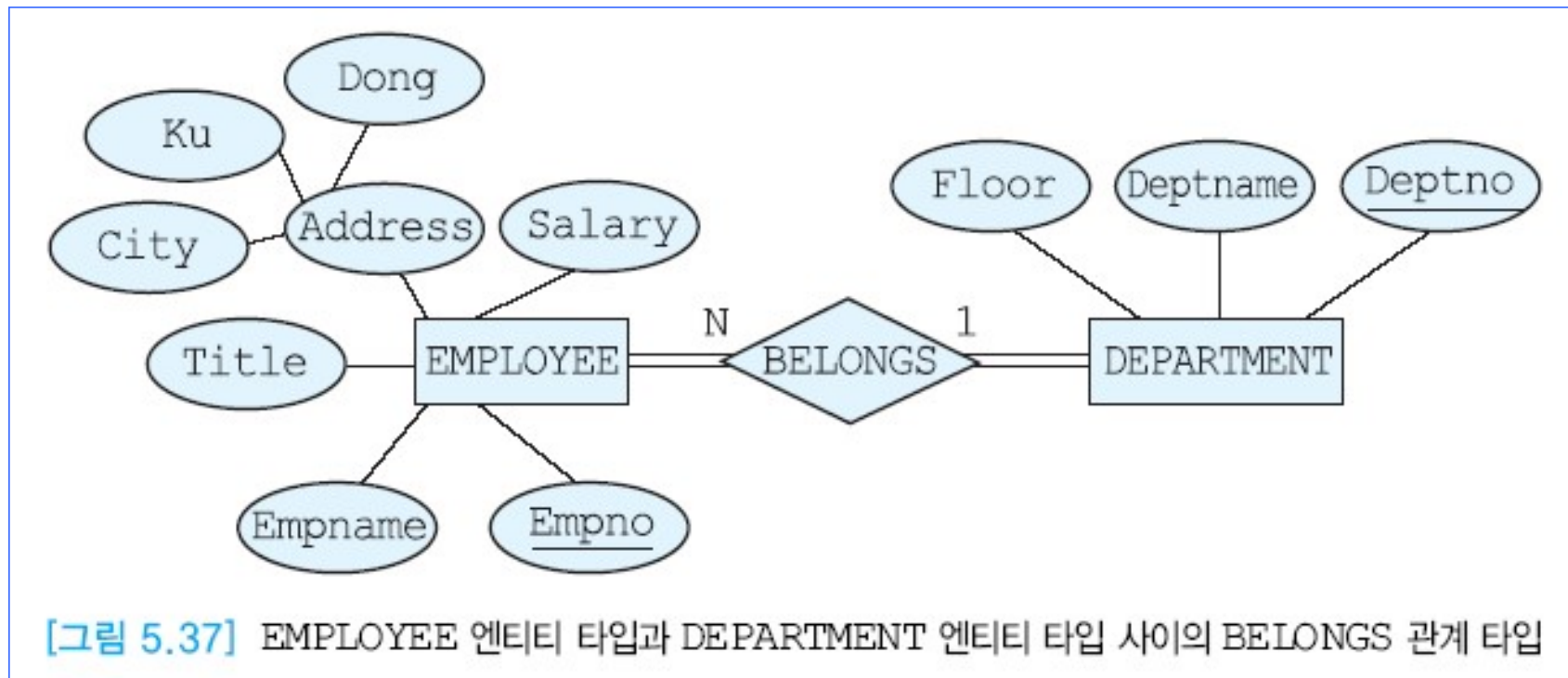
3. 데이터베이스 설계 예

□ 엔티티 타입 및 애트리뷰트들을 식별(계속)



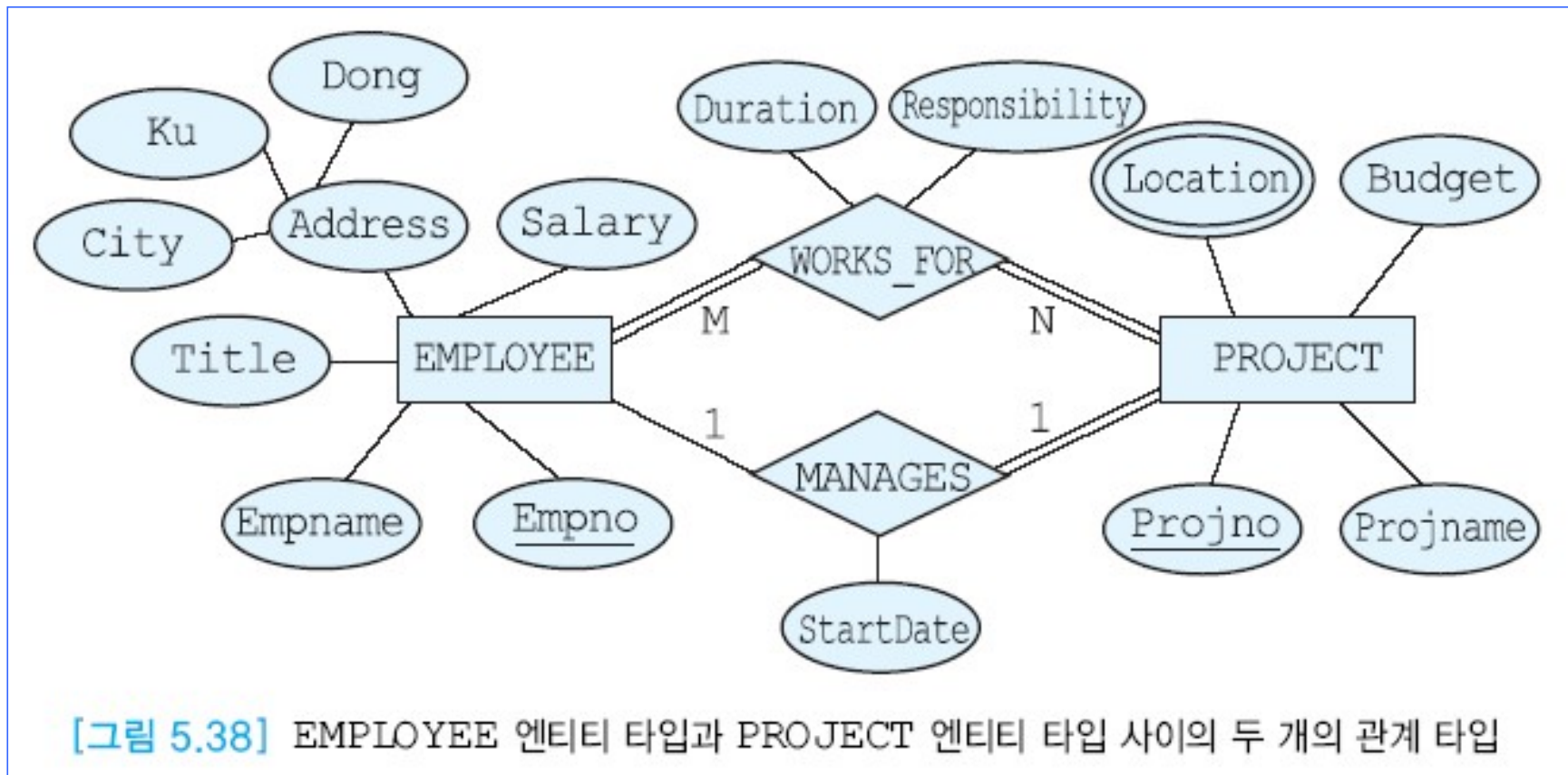
3. 데이터베이스 설계 예

□ 관계와 애트리뷰트들을 식별



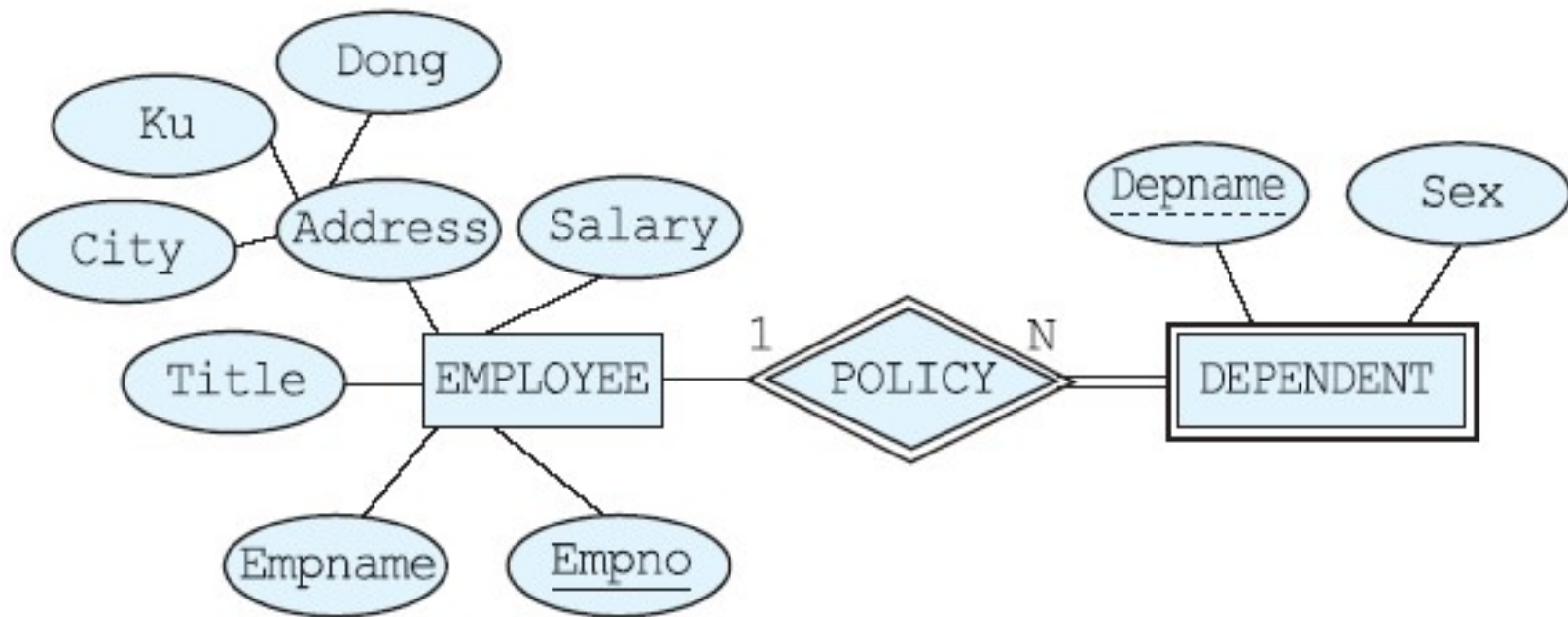
3. 데이터베이스 설계 예

□ 관계와 애트리뷰트들을 식별(계속)



3. 데이터베이스 설계 예

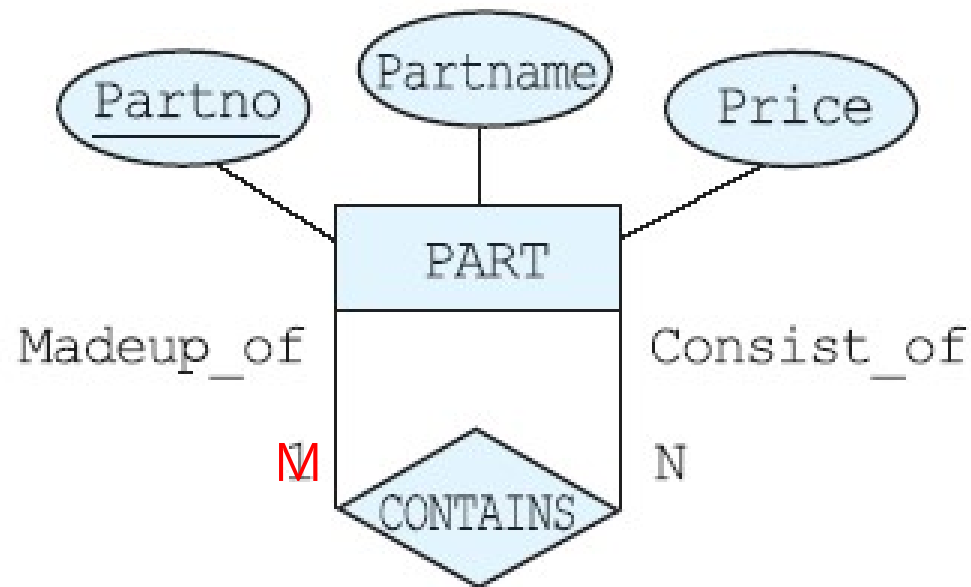
□ 관계와 애트리뷰트들을 식별(계속)



[그림 5.39] EMPLOYEE 엔티티 타입과 DEPENDENT 엔티티 타입 사이의 약한 관계 타입

3. 데이터베이스 설계 예

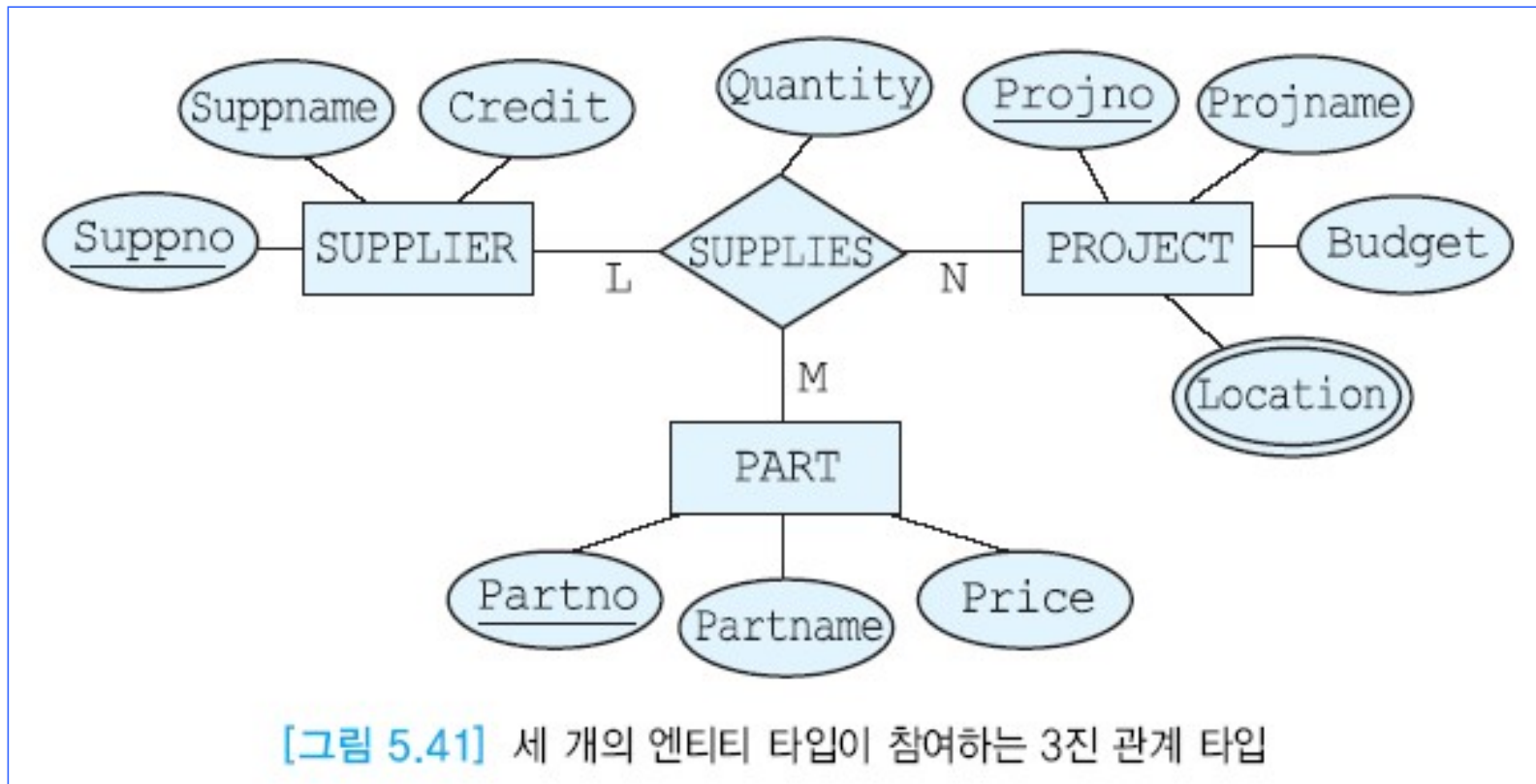
□ 관계와 애트리뷰트들을 식별(계속)



[그림 5.40] PART 엔티티 타입이 두 번 참여하는 순환적 관계 타입

3. 데이터베이스 설계 예

□ 관계와 애트리뷰트들을 식별(계속)



3. 데이터베이스 설계 예

□ 관계와 애트리뷰트들을 식별(계속)

